



Rapport Technique – Projet UrbanVibe

Projet Annuel B3

Sommaire

1. Introduction
2. Gestion de projet
3. Présentation fonctionnelle
4. Architecture technique
5. Frontend – UrbanVibe
6. Backend
7. Base de données
8. Interface & expérience utilisateur
9. Sécurité
10. Tests et validation
11. Limites, difficultés et axes d'amélioration
12. Glossaire

1. Introduction

UrbanVibe est une application web conçue dans le cadre du projet annuel B3-AL IW 2025. Elle a pour ambition de simplifier la recherche et la découverte d'événements locaux, en centralisant une offre qui reste aujourd'hui éparpillée sur différents sites et réseaux sociaux. Nous avons imaginé une plateforme claire, intuitive et agréable, qui permette à chacun de consulter, découvrir, et bientôt proposer des événements près de chez soi.

Pour développer UrbanVibe, nous avons adopté des pratiques et outils en phase avec les standards actuels du web, afin de nous rapprocher des conditions d'un projet professionnel. L'application s'appuie sur une **interface utilisateur** dynamique et adaptée aux différents formats d'écran (PC, mobile, tablette...), une **API backend** robuste et sécurisée, et une architecture pensée pour être facilement déployable et maintenable. Ces choix nous ont permis de mettre en pratique les compétences acquises durant notre formation tout en répondant aux exigences d'une expérience utilisateur fluide et moderne.

2. Gestion de projet

Pour assurer le bon déroulement et la coordination du projet, nous avons mis en place une organisation simple et efficace, inspirée des **méthodes agiles**. Nous avons choisi d'utiliser **Trello** comme outil principal pour planifier et suivre les tâches de développement de façon visuelle et collaborative, ainsi que **Git** pour le versionnement du code et la revue des contributions.

Sur **Trello**, nous avons structuré notre tableau sous forme de "**KANBAN**", en définissant quatre colonnes représentant les étapes du flux de travail :

Backlog : idées, fonctionnalités envisagées et tâches à venir (par exemple la publication sur les réseaux sociaux ou la carte interactive des lieux).

En cours : tâches actuellement en développement.

Tests unitaires / Intégration : tâches terminées mais en phase de vérification technique et fonctionnelle.

Terminé 🏁 : fonctionnalités finalisées et validées, qu'il s'agisse de **frontend**, **backend** ou configuration de l'environnement.

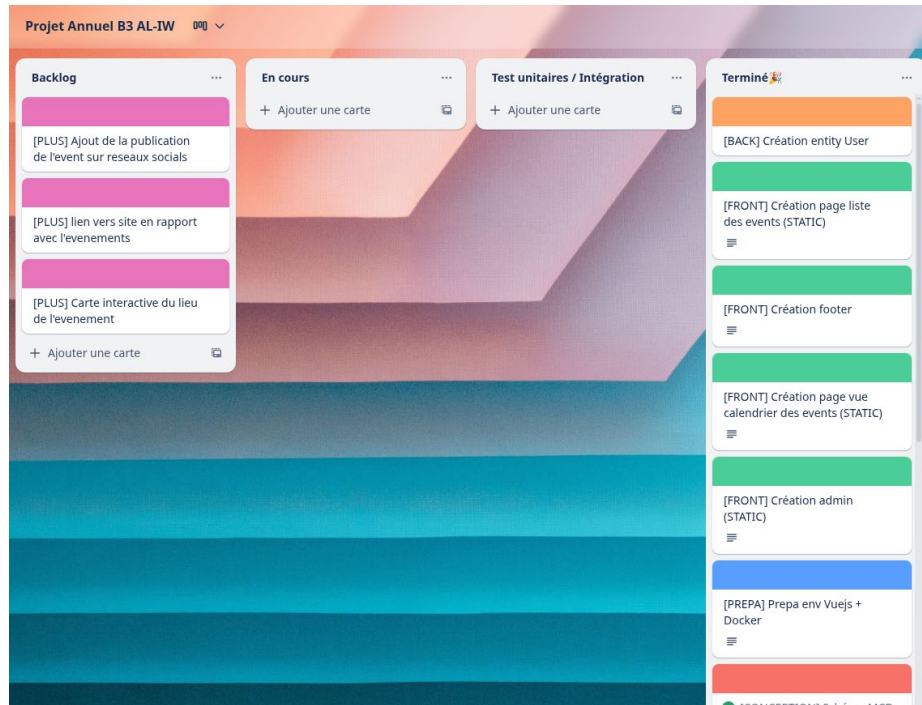


Fig. 2.1 - KANBAN sur Trello

En complément, nous avons utilisé [Git](#) pour gérer le code source : chaque fonctionnalité ou correctif était développé dans une [branche](#) dédiée, puis soumis via des merge requests pour relecture et validation avant intégration dans la [branche](#) principale. Ce [workflow](#) a permis de maintenir la qualité du code et d'éviter les conflits, tout en facilitant la collaboration au sein de l'équipe.

Cette organisation nous a permis de garder une vue d'ensemble sur l'avancement, de prioriser efficacement les tâches et d'identifier rapidement les points à traiter, tout en maintenant un rythme de développement structuré.

3. Présentation fonctionnelle

L'application UrbanVibe propose une expérience utilisateur centrée sur la consultation et l'interaction avec des événements locaux. Dès l'arrivée sur la page d'accueil, l'utilisateur se voit présenter une sélection d'événements sous forme de cartes visuelles, comportant le titre, une image illustrative, la date et un court descriptif. Cette présentation permet de se faire une idée rapide du contenu proposé, avec la possibilité de cliquer sur chaque carte pour accéder à une page de détails plus complète.

Les visiteurs peuvent également créer un compte ou se connecter via un formulaire d'authentification. Une fois identifiés, ils bénéficient de droits supplémentaires leur permettant de naviguer librement sur la plateforme, de s'inscrire à des événements et, à terme, d'en créer eux-mêmes. Le système est conçu pour être évolutif et s'adapter à de futures fonctionnalités comme la gestion de commentaires, les réactions ou les suggestions personnalisées.

La logique de l'application repose donc sur une expérience simplifiée mais riche, accessible à tous et rapidement compréhensible. Tout a été pensé pour rendre l'utilisation fluide, avec des actions claires, des retours visuels pertinents et un cheminement utilisateur logique.

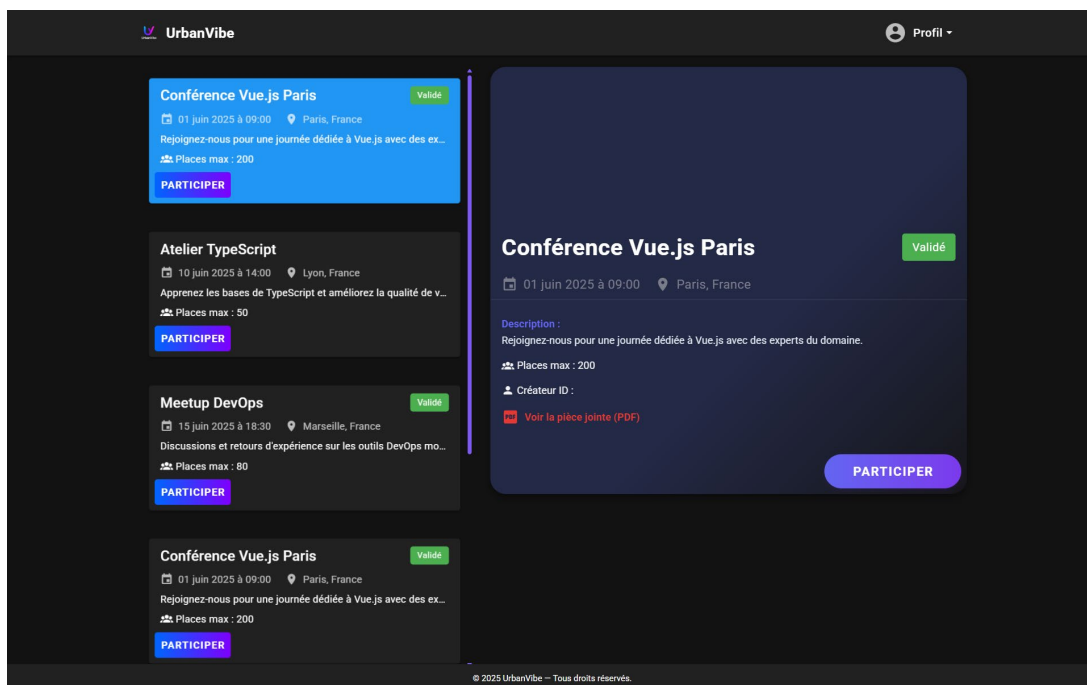


Figure 1 - Site UrbanVibe

4. Architecture technique

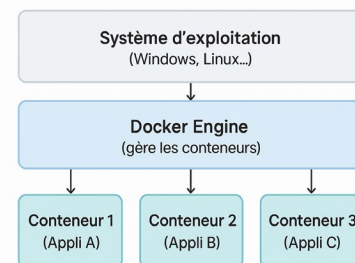
L'application UrbanVibe repose sur une architecture modulaire et bien structurée, pensée pour être maintenable, évolutive et proche des standards professionnels. Elle est composée de plusieurs couches indépendantes qui communiquent entre elles : le **frontend** (interface utilisateur), le **backend** (logique métier et **API**), la base de données et l'infrastructure de déploiement.

Dès le départ, nous avons fait le choix de technologies modernes et éprouvées :

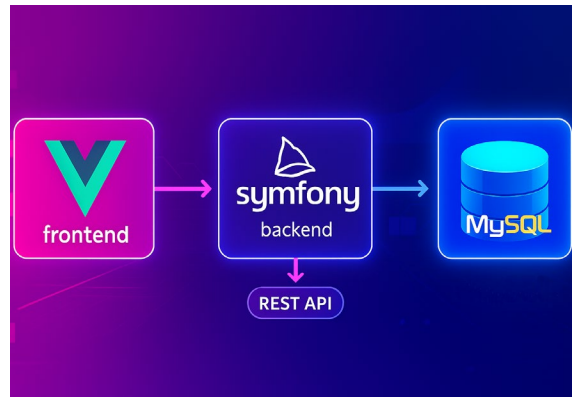
- Le **frontend** est développé avec le **Framework JavaScript Vue.js**, couplé à **TypeScript** pour améliorer la robustesse et la sécurité du code. L'interface graphique est mise en forme grâce à **Vuetify**, basé sur les principes du **Material Design**, afin d'offrir une expérience utilisateur harmonieuse et responsive. Le tout est servi par **Vite**, qui accélère les temps de compilation et de chargement pendant le développement.
- Le **backend** s'appuie sur le Framework **PHP Symfony**, qui apporte une structure solide grâce à son architecture **MVC** et son écosystème riche. Cet écosystème comprend par exemple le routage, qui permet de faire le lien entre une **URL** et une action à exécuter, la validation des données pour garantir leur conformité, ou encore la sérialisation, qui consiste à transformer des objets en formats lisibles comme **JSON** ou **XML**. Nous utilisons également **Doctrine** pour gérer la persistance des données, et **API Platform** pour exposer une **API REST** conforme aux bonnes pratiques.
- La **base de données** est une instance **MySQL**, gérée également par **Doctrine**, et intégrée dans l'environnement de développement pour plus de cohérence entre les environnements.
- Enfin, tous les services (**frontend**, **backend**, base de données) sont isolés et exécutés dans des conteneurs **Docker**.

Docker est une technologie qui permet d'empaqueter une application et toutes ses dépendances dans une "boîte" appelée conteneur. Chaque conteneur fonctionne de façon indépendante, ce qui garantit que l'application reste stable, reproductible et facilement déployable, quel que soit le système hôte utilisé (Windows, Linux, etc.).

Cela élimine les problèmes de compatibilité et facilite la gestion, le déploiement ou la mise à jour de l'ensemble des services.



Cette architecture nous permet de développer et tester chaque partie de façon autonome tout en garantissant l'intégration fluide de l'ensemble.



5. Frontend – UrbanVibe

Le **frontend** d'UrbanVibe constitue la vitrine de l'application : c'est la partie visible par les utilisateurs, celle avec laquelle ils interagissent au quotidien.

Pour le réaliser, nous nous sommes appuyés sur des technologies récentes telles que **Vue.js**, **TypeScript** et **Vuetify**, afin de garantir à la fois la robustesse du code et la qualité visuelle de l'application.

Ce choix technologique nous permet de proposer des fonctionnalités interactives et une navigation rapide, tout en assurant la cohérence et la maintenabilité du projet.

5.1 Objectifs

A - Objectifs fonctionnels

Les objectifs fonctionnels désignent l'ensemble des fonctionnalités concrètes que doit proposer UrbanVibe pour répondre aux besoins des utilisateurs.

Pour la version actuelle de l'application, les principales fonctionnalités sont :

Inscription et connexion sécurisée : permettre à chaque utilisateur de créer un compte et d'accéder à ses informations en toute sécurité.

Consultation des événements : offrir la possibilité de parcourir les événements disponibles autour de soi, via une interface visuelle sous forme de cartes claires et attractives.

Participation à un événement : permettre de s'inscrire ou signaler sa participation à un événement en un clic.

B - Objectifs d'expérience utilisateur (UX)

Au-delà des fonctionnalités, UrbanVibe vise à garantir une expérience utilisateur agréable, intuitive et accessible à tous.

Nos choix de conception **UX** ont été guidés par plusieurs objectifs :

Navigation fluide et intuitive : chaque page et action doivent être faciles à comprendre, sans apprentissage nécessaire, pour que l'utilisateur se concentre sur la découverte des événements et non sur l'utilisation de la plateforme.

Interface claire et responsive : grâce à l'utilisation du **Material Design** (principes de design définis par Google pour garantir lisibilité, cohérence, et adaptabilité sur tous les supports), l'application s'adapte à tous les types d'écrans (ordinateur, tablette, smartphone).

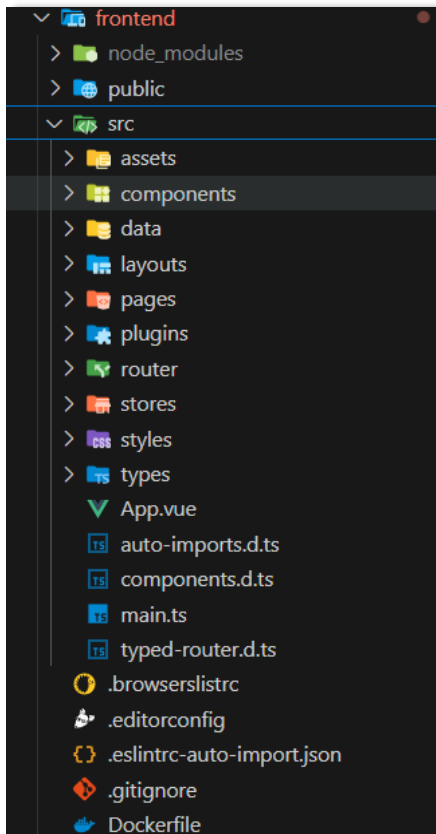
Attractivité visuelle : l'utilisation de cartes et d'éléments graphiques agréables permet de rendre la consultation plaisante.

Rapidité d'accès à l'information : tout est conçu pour que l'utilisateur trouve rapidement ce qu'il cherche, sans multiplication de clics inutiles.

Sécurité et confiance : l'inscription, la gestion des données personnelles et la participation à des événements se font dans un environnement sécurisé, respectant les standards actuels.

5.2. Architecture applicative

Pour structurer le *frontend*, nous avons adopté une architecture modulaire basée sur *Vue.js* 3, combinée avec *Vuetify* pour la couche graphique, et *Vite* pour la gestion du build. Voici un aperçu de l'organisation:



components : Regroupe les composants *Vue* réutilisables dans l'ensemble de l'application.

layouts : On y retrouve les structures de base pour les pages de l'application.

pages : Ce dossier contient les pages principales de l'application.

router : Contient la configuration du système de routage.

stores : Contient la logique de gestion d'état global avec "*Pinia*".

plugins : Utilisé pour initialiser et configurer certains outils de l'application comme "*Vuetify*", "*Axios*" ou d'autres bibliothèques.

styles : Fichiers CSS/SCSS globaux, gère " l'apparence " du site web.

types : Contient les définitions *TypeScript* partagées dans le projet.

assets : Ce dossier regroupe les fichiers statiques comme les images, les icônes...

5.3. Gestion des données et logique applicative

Nous avons opté pour une approche orientée objet, avec des interfaces strictement définies en *TypeScript* pour les entités clés (utilisateur, événement, participation...). Cela permet une meilleure lisibilité du code, une détection rapide des erreurs, et une collaboration fluide entre composants.

Les actions métier (comme récupérer les événements ou se connecter) sont centralisées dans des "*stores*" *Pinia*, garantissant une séparation claire entre la logique et l'affichage.

5.4. Dockerisation du frontend

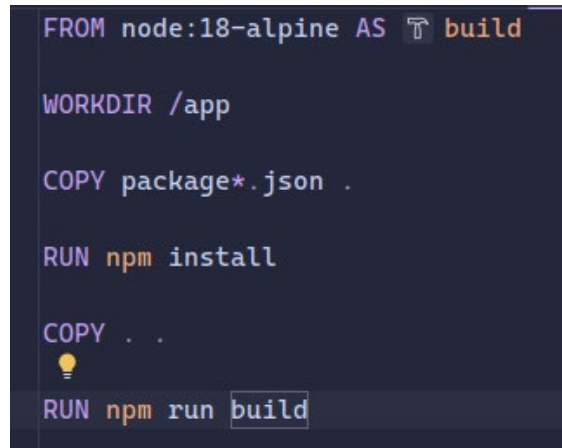
Pour garantir que l'application *frontend* puisse être exécutée dans un environnement contrôlé et indépendant de la machine de développement, nous avons créé une *image Docker* dédiée.

Cette image permet de compiler et servir le **frontend** de manière identique en développement et en production, tout en facilitant le déploiement.

Nous avons utilisé une image légère basée sur "Alpine Linux" et "Node.js 18".

Le processus consiste à installer les dépendances, compiler les assets statiques avec "**Vite**", puis exposer le répertoire "dist" pour être servi par un serveur web.

Voici le contenu du fichier "**Dockerfile**" utilisé :

A screenshot of a code editor with a dark background and light-colored text. The code is a Dockerfile with the following lines: FROM node:18-alpine AS build, WORKDIR /app, COPY package*.json ., RUN npm install, COPY . ., and RUN npm run build. The word 'build' in the last line is highlighted with a yellow box. There is a small lightbulb icon to the left of the last line.

```
FROM node:18-alpine AS build
WORKDIR /app
COPY package*.json .
RUN npm install
COPY . .
RUN npm run build
```

Fig. 5.4.1 - Code du fichier Dockerfile

Cette étape produit les fichiers statiques optimisés de l'application.

En production, ces fichiers peuvent ensuite être servis par un serveur web léger (par exemple **Nginx**), dans un second conteneur si besoin.

Cela nous assure une image de frontend propre et isolée, déployable facilement dans tout environnement **Docker**-compatible.

6. Backend

6.1. Introduction

Pour le développement du *backend* de notre application, nous avons utilisé le *Framework Symfony*, un *Framework PHP* moderne, robuste et orienté objet. *Symfony* repose sur l'architecture *MVC (Modèle – Vue – Contrôleur)*, qui permet une séparation claire des responsabilités, facilitant la maintenance, les tests et les évolutions futures du projet.

Le *pattern MVC* se compose de trois couches principales :

- **Modèle (Model)** : gère les données de l'application, les règles métiers, les entités et les interactions avec la base de données. Dans *Symfony*, cela correspond généralement aux entités *Doctrine* et aux repositories.
- **Vue (View)** : représente l'interface utilisateur. Elle est responsable de l'affichage des données sous forme de pages *HTML*. Dans *Symfony*, cela est pris en charge par les templates *Twig*.
- **Contrôleur (Controller)** : reçoit les requêtes de l'utilisateur, traite les données (via le modèle), et renvoie une réponse en appelant une vue. Il agit comme un chef d'orchestre.

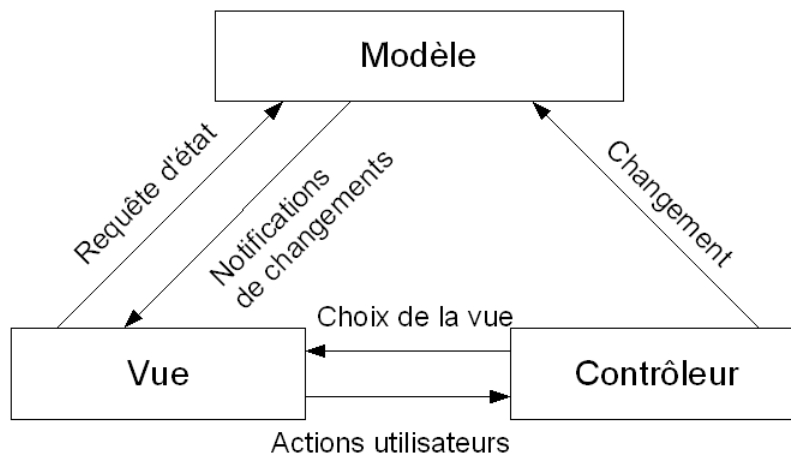


Fig. 6.1.1 - Schéma MVC

6.2. Entités

L'application repose sur un modèle de données bien structuré en entités **Doctrine**. Chaque entité représente **une table dans la base de données** et chaque instance de l'entité correspond à une **ligne** dans cette table. Doctrine **ORM** facilite la manipulation de ces objets en **PHP** tout en gérant leur persistance automatiquement. La figure 6.2.1 montre le schéma **MCD** (Modèle Conceptuel des Données) des entités de notre application.

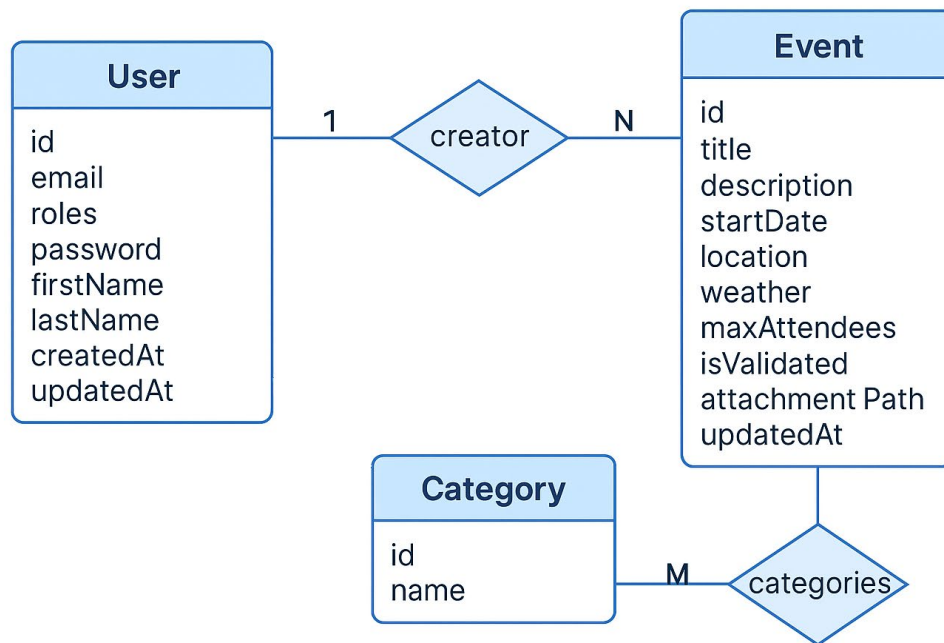


Fig. 6.2.1 – Schéma MCD des entités

```

class Category
{
    1 usage
    #[ORM\Id]
    #[ORM\GeneratedValue]
    #[ORM\Column]
    private ?int $id = null;

    2 usages
    #[ORM\Column(length: 255)]
    #[Assert\NotBlank()]
    #[Assert\Length(max: 255)]
    private ?string $name = null;

    /**
     * @var Collection<int, Event>
     */
    5 usages
    #[ORM\ManyToMany(targetEntity: Event::class, mappedBy: 'categories')]
    private Collection $events;
}

```

Fig. 6.2.2 – Code de l'entité "Category"

Le code ci-dessus montre l'entité **"Category"** en **PHP**.

En **Symfony**, les entités sont des **classes PHP** qui représentent les **tables de la base de données**.

Chaque propriété correspond à une colonne, et les **annotations (ou attributs)** permettent de configurer leur comportement.

Dans cet exemple, on observe plusieurs choses importantes :

- `#[ORM\Entity]` indique que cette classe est un **"entité Doctrine"**.
- `#[ORM\Id]`, `#[ORM\GeneratedValue]`, `#[ORM\Column]` définissent le champ **id** comme une **"clé primaire auto-incrémentée"**.
- Le champ `name` est de type `string`, avec des **"contraintes de validation"** :
- `#[Assert\NotBlank()]` : le nom ne peut pas être vide.
- `#[Assert\Length(max: 255)]` : longueur maximale de 255 caractères.
- `#[ORM\ManyToMany]` permet de définir une relation **"ManyToMany"** avec l'entité `Event` : une catégorie peut regrouper plusieurs événements, et un événement peut appartenir à plusieurs catégories.

6.3. DTO

Les **DTO**, ou objets de transfert de données, sont des classes simples utilisées pour **transporter des données d'un point à un autre** dans l'application, sans contenir de logique métier. Ils permettent de **décorréliser la structure des entités Doctrine** (souvent complexes ou sensibles) des données que l'on expose ou manipule, par exemple dans une **API** ou une interface utilisateur.

Dans notre projet, les **DTO** sont utilisés pour **structurer les réponses retournées par les contrôleurs** de manière claire, sécurisée et adaptée au besoin spécifique. Cela évite d'exposer directement les entités, qui peuvent contenir des données sensibles ou inutiles pour le client.

L'exemple suivant montre la classe "EventDto", utilisée pour transférer les informations liées à un événement.

```
class EventDto
{
    public int $id;
    public string $title;
    public string $description;
    public string $startDate;
    public string $location;
    public ?string $weather;
    public int $maxAttendees;
    public string $attachmentPath;
    1 usage
    public bool $isValidated;
    public array $attendees = [];

    public array $categories = [];
}
```

Fig. 6.3.1 - Code de EventDto

6.4. Sécurité du backend

Pour sécuriser l'accès à **l'API**, nous avons mis en place un système d'authentification basé sur **JWT (JSON Web Token)** grâce au bundle "**LexikJWTAuthenticationBundle**".

Ce système permet à un utilisateur de se connecter une fois, puis d'obtenir un **token signé** qu'il peut ensuite utiliser pour accéder aux **routes** sécurisées. Ce token est soit transmis via un "**header Authorization**", soit via un "**cookie sécurisé**".

6.5. Configuration JWT

Dans le fichier de configuration "lexik_jwt_authentication.yaml", on a défini les clés privées et publiques qui servent à signer et vérifier les **tokens**. On y précise également les différents moyens d'extraire le token lors des requêtes :

```
token_extractors:
  cookie:
    enabled: true
    name: BEARER
  authorization_header:
    enabled: true
  query_parameter:
    enabled: false
```

Cela signifie que l'application accepte les **tokens** transmis soit via un cookie nommé BEARER, soit via le *"header Authorization: Bearer <token>"*.

Configuration du système de sécurité

Dans le fichier security.yaml, on retrouve la configuration des **firewalls** (pare-feu), des **providers** (fournisseur) et des règles d'accès.

On utilise l'entité "**User**" comme provider, avec l'email comme identifiant. L'authentification se fait via la route /api/login_check, en utilisant un système de login JSON.

Les **firewalls** suivants sont définis :

- login : s'occupe uniquement de l'authentification. Il est "**stateless**" et ne garde pas de session.
- api : protège toutes les routes /api après la connexion. Il utilise le **token JWT** pour vérifier l'identité de l'utilisateur.
- main : protège la partie /admin, uniquement accessible aux administrateurs.

Enfin, les **règles d'accès (access_control)** permettent de définir les droits selon les rôles :

- Les routes /api/login_check, /api/login, /api/register et /api/events sont publiques.
- Toutes les autres routes /api nécessitent que l'utilisateur soit connecté.
- Les routes /admin sont réservées aux utilisateurs ayant le rôle ROLE_ADMIN.

6.6. Dockerisation

Pour simplifier le déploiement et garantir un environnement de développement homogène, l'application a été **conteneurisée avec Docker**. Cette approche permet de regrouper l'ensemble des dépendances (**PHP**, **Composer**, extensions, etc.) dans une **image unique**, facilement exécutable sur n'importe quelle machine.

Le fichier "Dockerfile" définit la configuration du conteneur principal qui exécute **PHP** avec **FPM** (FastCGI Process Manager). Il sert de base à la création de **l'image Docker** pour notre projet **Symfony**.

```
1 FROM php:8.2-fpm
2
3 RUN apt-get update && apt-get install -y \
4     git unzip libzip-dev zip libpng-dev libonig-dev libxml2-dev mariadb-client \
5     && docker-php-ext-install pdo pdo_mysql zip
6
7 COPY --from=composer:latest /usr/bin/composer /usr/bin/composer
8
9 WORKDIR /var/www/html
10
11 COPY . .
12
13 RUN composer install --no-interaction --prefer-dist
14
15 COPY docker/entrypoint.sh /usr/local/bin/entrypoint.sh
16
17 ENTRYPOINT ["/usr/local/bin/entrypoint.sh"]
```

Le Dockerfile utilise l'image officielle "php:8.2-fpm" comme point de départ. Plusieurs instructions y sont définies :

- RUN apt-get update ... : cette ligne installe les dépendances nécessaires pour faire fonctionner Symfony (bibliothèques ZIP, image, base de données, etc.).
- docker-php-ext-install ... : installation des extensions **PHP** indispensables comme pdo, pdo_mysql, zip, etc.
- COPY --from=composer:latest ... : récupération de **Composer**, l'outil de gestion des dépendances PHP, depuis l'image officielle.
- WORKDIR /var/www/html : définition du répertoire de travail à l'intérieur du conteneur.
- COPY . . : copie des fichiers du projet dans le conteneur.
- RUN composer install ... : installation des dépendances **Symfony** du projet.
- COPY docker/entrypoint.sh ... : copie du script de démarrage personnalisé dans le conteneur.
- ENTRYPOINT [...] : définition du point d'entrée du conteneur, qui exécute le script "entrypoint.sh".

6.7. Administration des événements

Une interface d'administration a été mise en place afin de permettre aux utilisateurs ayant un rôle administrateur de gérer les événements présents dans la base de données. Cette section a été pensée pour être à la fois simple d'utilisation et fonctionnelle.

L'administrateur peut accéder à un tableau de bord listant tous les événements, classés par ordre décroissant d'identifiant. Depuis cette interface, il peut consulter les détails d'un événement, modifier ses informations, le valider, le refuser ou encore le supprimer définitivement. Un bouton permet également de créer un nouvel événement depuis un formulaire dédié.

Chaque ligne du tableau présente les informations principales de l'événement, telles que le titre, la date de début, la ville, la météo prévue, le nom du créateur, ainsi qu'un indicateur visuel indiquant si l'événement est validé ou non. Des boutons d'action permettent ensuite de visualiser, éditer, valider/refuser ou supprimer l'événement.

Lorsqu'un administrateur consulte un événement en détail, toutes les informations sont affichées, y compris les participants inscrits, la description, ainsi qu'un éventuel fichier joint (image ou document PDF). L'interface s'adapte automatiquement selon le type de fichier : il peut être affiché directement ou proposé au téléchargement.

L'interface d'édition et de création d'événement repose sur le même formulaire, et permet à l'administrateur de renseigner ou modifier l'ensemble des champs nécessaires. Lors de la création, les événements ne sont pas validés par défaut. Le système de validation repose sur un simple bouton qui permet d'activer ou de désactiver la visibilité de l'événement côté utilisateur.

Enfin, des protections comme les **jetons CSRF** sont en place pour sécuriser les actions sensibles comme la validation ou la suppression d'un événement.

Administration des événements

Créer un événement

ID	Créé par	Titre	Début	Ville	Météo prévue	Validée ?	Actions			
#14	John2	Mon premier événement	10/07/2025	Paris	Ciel clair	Non	Voir	Modifier	Valider	Supprimer
#12	John0	Event with image	05/07/2025	Valence		Oui	Voir	Modifier	Refuser	Supprimer
#10	John4	Événement 9	10/07/2025	Ville 9		Oui	Voir	Modifier	Refuser	Supprimer
#9	John1	Événement 8	09/07/2025	Ville 8		Oui	Voir	Modifier	Refuser	Supprimer
#8	John1	Événement 7	08/07/2025	Ville 7		Oui	Voir	Modifier	Refuser	Supprimer
#7	John1	Événement 6	07/07/2025	Ville 6		Oui	Voir	Modifier	Refuser	Supprimer
#6	John0	Événement 5	06/07/2025	Ville 5		Oui	Voir	Modifier	Refuser	Supprimer
#5	John4	Événement 4	05/07/2025	Ville 4		Oui	Voir	Modifier	Refuser	Supprimer
#4	John3	Événement 3	04/07/2025	Ville 3		Oui	Voir	Modifier	Refuser	Supprimer
#3	John1	Événement 2	03/07/2025	Ville 2		Oui	Voir	Modifier	Refuser	Supprimer
#2	John4	Événement 1	02/07/2025	Ville 1		Oui	Voir	Modifier	Refuser	Supprimer
#1	John2	Événement 0	01/07/2025	Ville 0		Oui	Voir	Modifier	Refuser	Supprimer

Fig. 6.7.1 – Administration des événements

Dans cette interface, l’affichage dynamique des événements repose sur l’utilisation du moteur de templates *Twig*. La variable "events", transmise par le contrôleur, contient l’ensemble des événements récupérés depuis la base de données. Elle est parcourue dans la vue à l’aide d’une "boucle for", ce qui permet d’afficher automatiquement une ligne dans le tableau pour chaque événement.

Le contrôleur associé injecte cette variable dans la vue en appelant la méthode "findBy" du *repository*, qui retourne tous les événements triés par ordre décroissant d’identifiant. Cette logique est encapsulée dans l’action "index()" du contrôleur "AdminEventController".

Chaque événement affiché dans le tableau contient ses principales informations : titre, date, lieu, créateur, météo prévue, et un indicateur de validation. Plusieurs actions sont disponibles sur chaque ligne : voir les détails, modifier l’événement, changer son statut (valider/refuser), ou le supprimer. Ces actions sont directement reliées aux routes définies dans le contrôleur.

L’ensemble du rendu est ainsi entièrement **dynamique**, basé sur les données en base, et réactif aux actions effectuées par l’administrateur. Cela permet une gestion simple, rapide et efficace des événements via l’interface d’administration.

```
#[Route('', name: 'index', methods: ['GET'])]
public function index(): Response
{
    return $this->render(
        view: 'admin/event/index.html.twig',
        [
            'events' => $this->eventRepository->findBy([], ['id' => 'DESC']),
        ]
    );
}
```

Fig. 6.7.2 – Code de la méthode `index()` du contrôleur "AdminEventController"

La capture ci-dessus montre l'action "`index()`" du contrôleur d'administration des événements. Cette méthode est liée à la route `/admin/event` grâce à l'attribut `#[Route]`. Elle est appelée lorsqu'un administrateur accède à la page principale de gestion des événements.

La méthode utilise le composant "`render()`" pour afficher le template **Twig** `admin/event/index.html.twig`. Ce dernier reçoit en paramètre une variable nommée `events`.

Cette variable est alimentée par un appel au **repository** de l'entité `Event`. La méthode "`findBy([], ['id' => 'DESC'])`" permet de récupérer tous les événements de la base de données, sans condition de filtre, mais en les triant par identifiant décroissant. L'objectif est d'afficher en haut de la liste les événements les plus récents.

Cette variable `events` est ensuite exploitée dans le template **Twig** pour générer dynamiquement le tableau d'administration, comme expliqué précédemment.

6.8. Service API météo

Pour améliorer l'expérience de gestion des événements dans notre application, nous avons mis en place un système automatique de récupération de la météo. L'idée était de permettre à l'administrateur, au moment où il crée ou modifie un événement, de connaître rapidement les prévisions météorologiques pour le lieu et la date choisis, sans avoir à sortir de l'interface.

Pour cela, nous avons développé un service que nous avons appelé "WeatherService". Ce service repose sur l'appel à deux **API** externes. La première, "Nominatim", nous permet de convertir un lieu (comme le nom d'une ville) en coordonnées GPS. La seconde, "Open-Meteo", fournit les prévisions météo en fonction de ces coordonnées, pour une date précise. Le service traite ensuite la réponse de **l'API** pour en extraire un code météo, qui est converti en description lisible par l'utilisateur (par exemple : « Ciel clair », « Neige forte » ou « Brouillard »).

La méthode principale de ce service, "getWeather", se charge de toute cette logique. Elle commence par géocoder la localisation, puis envoie une requête vers **l'API** météo. Une fois la réponse reçue, elle extrait le code correspondant à la météo prévue et le convertit en texte clair. Si le lieu est inconnu ou si **l'API** ne renvoie aucun résultat, la méthode retourne simplement "null".

```
15 public function getWeather(string $location, \DateTimeInterface $date): ?string
16 {
17     $coords = $this->geocodeLocation($location);
18
19     if (!$coords) {
20         return null;
21     }
22
23     $isoTime = $date->format('Y-m-d\TH:00:00');
24
25     $url = 'https://api.open-meteo.com/v1/forecast';
26     $response = $this->client->request('GET', $url, [
27         'query' => [
28             'latitude' => $coords['lat'],
29             'longitude' => $coords['lon'],
30             'hourly' => 'weathercode',
31             'start' => $isoTime,
32             'end' => $isoTime,
33             'timezone' => 'Europe/Paris',
34         ],
35     ]);
36
37     $data = $response->toArray(false);
38
39     if (!isset($data['hourly']['weathercode'][0])) {
40         return null;
41     }
42
43     $code = $data['hourly']['weathercode'][0];
44
45     return $this->mapWeatherCodeToText($code);
46 }
```

Fig. 6.8.1 - Code de la méthode getWeather()

Ce service est exploité de manière dynamique dans le formulaire d'administration des événements. Grâce à "**Stimulus**", un petit **Framework JavaScript**, nous avons pu connecter les champs « lieu » et « date » à une action **JavaScript**. Dès que l'un de ces champs est modifié, une requête est envoyée à une route **Symfony** dédiée, qui appelle le "WeatherService" et renvoie une réponse **JSON** contenant la météo. Cette donnée est ensuite automatiquement affichée dans un champ en lecture seule du formulaire.

Pour rendre cela possible, nous avons ajouté des attributs **HTML** "data-*" aux champs concernés dans le formulaire **Symfony**. Ils permettent à **Stimulus** de cibler les bons éléments et de déclencher l'appel **API** au bon moment. Ce fonctionnement fluide et discret améliore nettement l'ergonomie du **back-office** sans ajouter de complexité pour l'administrateur.

7. Base de données

7.1. Introduction

L'application utilise "**MariaDB**" comme système de gestion de base de données relationnelle. **MariaDB** est une solution open source compatible avec **MySQL**, réputée pour sa stabilité, ses performances et sa facilité d'intégration avec **Symfony** via **Doctrine ORM**.

Dans le cadre de ce projet, la base de données est "**conteneurisée avec Docker**", ce qui permet de garantir un environnement de travail identique sur toutes les machines, que ce soit en développement ou en production. La configuration du service **MariaDB** est définie dans le fichier "compose.yml", où le conteneur expose le **port** "3306" et utilise un volume pour la persistance des données. Des variables d'environnement (utilisateur, mot de passe, nom de la base) sont également définies afin de faciliter la connexion avec l'application **Symfony**.

Grâce à cette architecture, l'environnement peut être lancé rapidement avec une seule commande "**docker compose up**", sans avoir besoin d'installer **MariaDB** localement sur l'hôte. **Symfony** se connecte ensuite automatiquement à cette base via les paramètres renseignés dans le fichier "**.env**".

Doctrine ORM se charge de gérer les entités **PHP** et de les synchroniser avec les tables de la base de données. Les "**migrations**" permettent d'automatiser la création ou la mise à jour du schéma. Cela assure une bonne évolutivité de la structure tout au long du projet.

7.2. Schéma relationnel

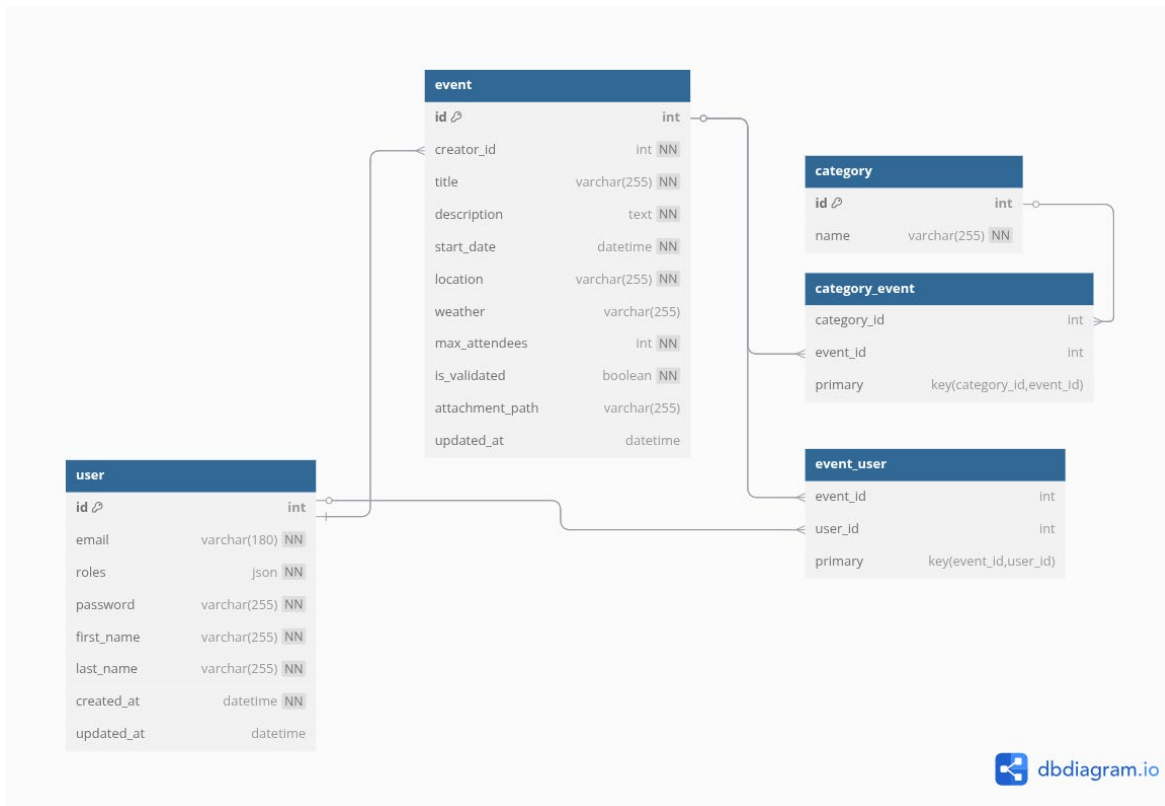


Fig. 7.2.1 - Schéma relationnel de l'application

7.3. Dockerisation de la BDD

```
db:
  image: mariadb:11
  environment:
    MYSQL_ROOT_PASSWORD: ${MYSQL_ROOT_PASSWORD}
    MYSQL_DATABASE: ${MYSQL_DATABASE}
    MYSQL_USER: ${MYSQL_USER}
    MYSQL_PASSWORD: ${MYSQL_PASSWORD}
  ports:
    - "3310:3306"
  volumes:
    - db-data:/var/lib/mysql
```

Fig. 7.3.1 - Configuration du service db

La capture d'écran ci-dessus montre la définition du service db dans le fichier docker-compose.yml. Ce service correspond au conteneur **MariaDB** utilisé pour gérer la base de données de l'application.

Le conteneur est basé sur l'image officielle "**mariadb:11**", ce qui garantit un environnement stable, maintenu et compatible avec les outils actuels. Plusieurs variables d'environnement sont définies afin d'initialiser correctement la base dès le démarrage du conteneur. Il s'agit notamment du mot de passe root (MYSQL_ROOT_PASSWORD), du nom de la base de données (MYSQL_DATABASE), ainsi que des identifiants d'un utilisateur classique (MYSQL_USER et MYSQL_PASSWORD). Ces valeurs sont généralement définies dans un fichier ".env" situé à la racine du projet.

Le **port "3306"** exposé par **MariaDB** à l'intérieur du conteneur est redirigé vers le **port "3310"** de la machine hôte. Cela permet de se connecter à la base de données localement à l'aide d'un client MySQL/MariaDB, sans interférer avec d'autres services potentiels déjà présents sur le **port "3306"**.

Un volume nommé db-data est également monté sur le répertoire /var/lib/mysql du conteneur. Ce volume assure la **persistance des données** : même si le conteneur est supprimé ou redémarré, le contenu de la base est conservé sur la machine hôte. Cela permet de travailler avec un environnement isolé tout en maintenant les données intactes d'une session à l'autre.

8. Interface & UX

8.1. UX

Pour concevoir une interface centrée sur l'utilisateur, nous avons utilisé plusieurs outils clés : une **Empathy Map** pour mieux comprendre les besoins, émotions et motivations des utilisateurs, un **User Flow** afin de visualiser clairement le parcours que l'utilisateur suit pour accomplir ses tâches sur le site et une **Bubble Map** qui nous a aidés à organiser visuellement les différentes fonctionnalités et leur importance relative.

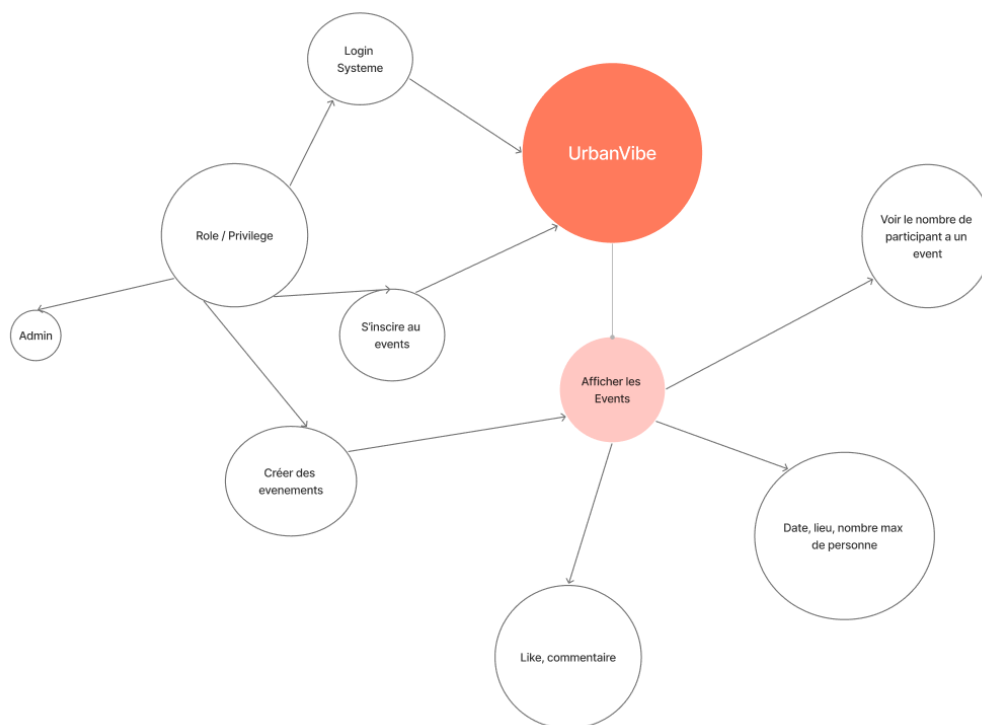


Fig. 2 - Bubble Map

8.2. Interface

Nous avons mis l'accent sur l'expérience utilisateur. Dès les premières maquettes, on voulait quelque chose de clair, moderne, accessible et agréable à utiliser. Le thème sombre s'est imposé naturellement, avec des contrastes bien dosés, une typographie lisible, des espaces bien organisés...

Fig. 3 - Première maquette

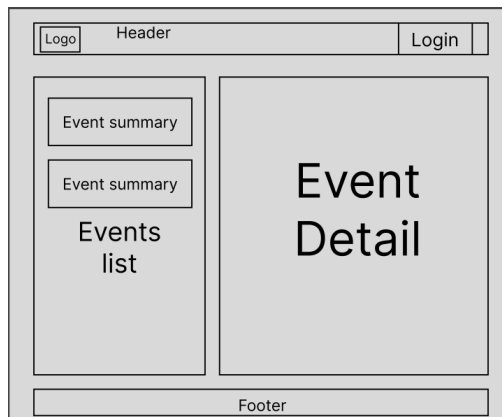
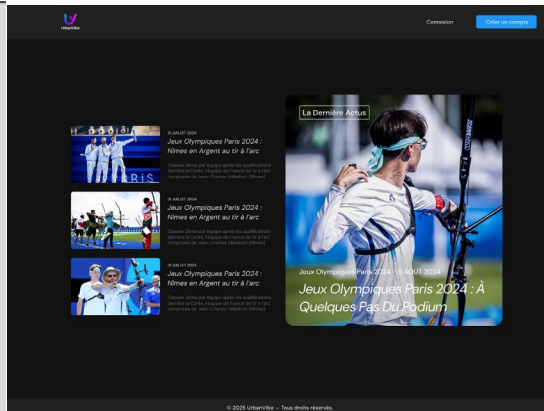


Fig. 4.2.2 - Maquette améliorée



Les composants comme *EventCard* ou *EventDetails* permettent une navigation intuitive. L'utilisateur sait toujours où il est. Grâce à **Vuetify**, l'app s'adapte super bien aux écrans mobiles, ce qui était important pour nous.

8.3. Référencement

Le site dispose de éléments renforçant le référencement. Par exemple, sur le fichier de base du site "*index.html*", il dispose de "**balises**" type "*meta*" permettant au site d'avoir une intégration sur des réseaux sociaux.

```
<!DOCTYPE html>
<html lang="fr">
<head>
  <title>UrbanVibe - Événements et communauté</title>
  <meta name="description" content="Découvrez, participez et organisez des événements sur UrbanVibe. Rejoignez la communauté dès maintenant !">
  <meta name="keywords" content="événements, UrbanVibe, communauté, musique, sorties, soirées, participations">
  <meta name="author" content="UrbanVibe Team">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <meta http-equiv="Content-Type" content="text/html; charset=UTF-8">

  <meta property="og:title" content="UrbanVibe - Événements et communauté" />
  <meta property="og:description" content="Découvrez, participez et organisez des événements sur UrbanVibe. Rejoignez la communauté dès maintenant !" />
  <meta property="og:image" content="urbanvibe.png" />
  <meta property="og:url" content="https://urbanvibe.com" />
  <meta property="og:type" content="website" />

  <meta name="twitter:card" content="summary_large_image">
  <meta name="twitter:title" content="UrbanVibe - Événements et communauté">
  <meta name="twitter:description" content="Découvrez, participez et organisez des événements sur UrbanVibe.">
  <meta name="twitter:image" content="/urbanvibe.png">
</head>
<body>
  <div id="app"></div>
  <script type="module" src="/src/main.ts"></script>
</body>
</html>
```

Google

Réseaux sociaux

Twitter (X)

Fig. 5 - Fichier principal du site avec les balises meta

Le site utilise également l'attribut "alt" sur les **balises** d'image (). Cet attribut répond aux enjeux d'accessibilité, notamment pour les personnes malvoyantes. Il contribue aussi au référencement **SEO** sur les moteurs de recherche. Enfin, il s'affiche en cas d'erreur si l'image ne peut pas être chargée sur la page.

Statistiques du site sur ses performances, son accessibilité, ses bonnes pratiques et son **SEO**.

```
✓ EventCardDetail.vue frontend\src\components  
  alt="Image associée à l'événement"  
✓ EventDetails.vue frontend\src\components  
  alt="UrbanVibe Logo"  
✓ UrbanHeader.vue frontend\src\components  
  alt="UrbanVibe Logo"
```

Fig. 8.3.2 - Attribut alt sur les balises

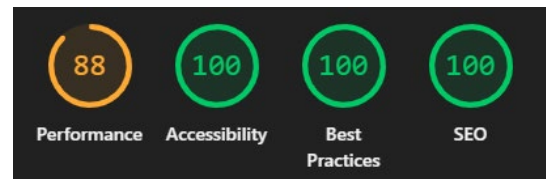


Fig. 8.3.3 - Statistiques du site

9. Sécurité

La sécurité de l'application est un point clé, pris en charge principalement côté **backend** grâce aux fonctionnalités offertes par **Symfony**. Notre objectif est de contrôler précisément qui peut accéder à quelles ressources, en fonction de son rôle dans le système (ROLE_ADMIN, ou simple utilisateur).

9.1. Authentification

Nous avons mis en place un système d'authentification basé sur des "**JSON Web Tokens (JWT)**", via le "**bundle *lexik/jwt-authentication-bundle***".

Ce mécanisme, bien adapté aux applications **SPA** comme UrbanVibe, évite la gestion classique des sessions côté serveur en utilisant des jetons signés envoyés dans le "**header Authorization**" ou dans un **cookie sécurisé**.

Un utilisateur authentifié obtient un **JWT** lors de sa connexion, qu'il peut ensuite utiliser pour accéder aux routes sécurisées de **l'API**.

Ce choix apporte simplicité et performance, mais nous restons conscients des limites de **JWT**: en particulier, il est important de bien contrôler la durée de validité des **jetons** et d'envisager des mécanismes de révocation en cas de besoin (par exemple en stockant une liste noire en base).

9.2. Autorisation et gestion des rôles

Au-delà de l'authentification, **Symfony** nous permet de gérer finement l'**autorisation** grâce à son système de **rôles** et de **contrôle d'accès**.

Dans le fichier "**security.yaml**", nous avons défini des règles précises :

Les routes publiques, comme /api/login_check, /api/register ou /api/events, restent accessibles sans authentification.

Toutes les autres routes /api nécessitent un **JWT** valide et un utilisateur authentifié.

Les routes /admin sont protégées et réservées aux utilisateurs ayant le rôle ROLE_ADMIN.

Certaines actions de gestion d'événements peuvent être restreintes aux utilisateurs ayant le rôle ROLE_ORGANISATEUR.

Ces règles garantissent une séparation claire des permissions et facilitent la maintenance en cas d'évolution des besoins en matière de sécurité.

9.3 Évolutions possibles

Pour renforcer encore la sécurité, plusieurs pistes d'amélioration sont envisagées :

- Révocation des *tokens JWT* via une liste noire pour invalider les *jetons* compromis.
- Mise en place d'une authentification à deux facteurs pour les administrateurs.
- Surveillance active des tentatives de connexion suspectes pour détecter les attaques par force brute.

10. Tests et validation

Pour l'instant, les tests ont été faits manuellement. On a vérifié chaque fonctionnalité au fur et à mesure du développement. Des tests automatisés sont prévus.

11. Limites, difficultés et améliorations possibles

Au cours du développement du projet, nous avons rencontré certaines difficultés techniques et organisationnelles, ainsi que des limites inhérentes à nos choix architecturaux. Ces constats nourrissent également des pistes d'amélioration pour les évolutions futures.

11.1. Limites et difficultés

Projet global

- La mise en place d'une configuration **Docker** adaptée et sécurisée pour le **frontend** et le **backend** a été plus longue que prévu, notamment pour trouver des images légères, compatibles et faciles à maintenir.
- La communication entre le **frontend** et le **backend** (gestion des **CORS**, configuration des URLs) a nécessité plusieurs itérations avant d'être pleinement fonctionnelle.

Frontend

- Définir un style graphique cohérent pour tout le site a été un challenge, et la barre de navigation a demandé plusieurs essais pour être à la fois esthétique et ergonomique.
- La configuration et l'utilisation d'outils de qualité de code (**linter**, formateur automatique) ont pris du temps, mais se sont avérés essentiels pour garantir la cohérence du code.

Backend

- La séparation des responsabilités entre **Symfony** et le **frontend** pour la gestion des **routes** (et notamment la cohabitation avec **Vue.js** sur le serveur web) a généré quelques confusions au début.
- L'architecture **MVC**, choisie pour sa simplicité, montre ses limites dans un contexte plus complexe : le risque d'encombrer les contrôleurs avec de la logique métier (au lieu de la placer dans des services) peut rendre le code difficile à maintenir à long terme. Pour y remédier, il faudra envisager de structurer davantage la logique applicative en dehors des contrôleurs.

11.2. Améliorations possibles

Par manque de temps, plusieurs améliorations restent à prévoir et pourraient être intégrées lors d'une prochaine itération du projet :

- **Qualité et maintenance**
 - Ajouter des **tests unitaires** et **d'intégration**, côté **frontend** comme **backend**.
 - Effectuer un **refactoring** général du code pour le rendre plus lisible, modulaire et maintenable.
 - Optimiser la configuration **Docker** pour accélérer le développement local et améliorer la rapidité des builds.
- **Fonctionnalités**
 - Permettre aux utilisateurs de proposer et gérer leurs propres événements directement depuis la plateforme.
 - Déplacer le **backoffice** actuellement intégré dans **Symfony** vers le **frontend Vue.js**, pour une expérience unifiée.
- **Expérience utilisateur**
 - Améliorer encore la responsivité et l'ergonomie de l'interface sur mobile et tablettes.
 - Enrichir le style graphique pour renforcer l'identité visuelle de la plateforme.
- **Architecture**
 - Repenser certaines parties du **backend** pour éviter les limites du **MVC** classique, en isolant davantage la logique métier dans des services et en évitant le mélange des responsabilités.

12. Glossaire

API	Interface qui permet à différentes applications de communiquer entre elles en échangeant des données.
API REST	Interface de programmation qui permet à différentes applications de communiquer entre elles via le protocole HTTP, en suivant des principes architecturaux simples, lisibles et standardisés.
Axios	Bibliothèque JavaScript permettant d'effectuer des requêtes HTTP depuis un navigateur ou un environnement Node.js, utilisée pour communiquer avec des APIs et récupérer ou envoyer des données.
Backend	Partie du projet qui ne sera pas visible pas l'utilisateur. Il communique avec le Frontend et traitera les actions avec la base de donnée
Backoffice	Interface d'administration réservée aux gestionnaires du site, permettant de modifier ou gérer les contenus et les utilisateurs.
Balise HTML	Éléments de syntaxe entourés de chevrons (< >) utilisés pour structurer et afficher le contenu d'une page web, comme les titres, paragraphes, images ou liens.

Branche GIT	Copies isolées du code principal dans un dépôt Git, permettant de développer de nouvelles fonctionnalités ou corriger des bugs sans affecter la version stable du projet.
Bubble Map	une carte visuelle où l'on regroupe et relie des idées ou fonctionnalités sous forme de bulles, ce qui facilite l'organisation et la hiérarchisation du contenu.
Classe PHP	Structures dans le langage PHP qui permettent de définir des objets en regroupant des propriétés (données) et des méthodes (fonctions) pour organiser et réutiliser le code de manière orientée objet.
Composer	Gestionnaire de dépendances pour PHP qui permet d'installer, mettre à jour et gérer facilement les bibliothèques et packages nécessaires à un projet.
Cookie sécurisé	Fichier stocké par le navigateur contenant des données, transmis uniquement via une connexion HTTPS, afin de garantir la confidentialité et l'intégrité des informations échangées
CORS	Mécanisme de sécurité qui permet à un site web d'autoriser ou de restreindre les ressources demandées depuis un autre domaine, protégeant ainsi contre les accès non autorisés entre sites.
Docker	Outil qui permet d'exécuter des applications dans des conteneurs isolés et portables.
Doctrine (PHP)	Bibliothèque qui facilite la gestion de base de données en PHP via un système de mapping objet-relationnel (ORM).
DTO	Objet servant à transporter des données entre différentes couches d'une application sans contenir de logique métier, afin de simplifier les échanges et structurer l'information.
Empathy Map	un outil qui permet de se mettre à la place de l'utilisateur pour identifier ce qu'il pense, ressent, voit et entend, afin de mieux répondre à ses attentes.
Firewalls	Dispositifs ou logiciels de sécurité qui contrôlent et filtrent le trafic réseau entrant et sortant pour protéger un système contre les accès non autorisés et les attaques.
FPM	FastCGI Process Manager, Gestionnaire de processus PHP optimisé pour gérer efficacement plusieurs requêtes simultanées, améliorant la performance des sites web utilisant PHP.
Framework	Ensemble d'outils et de règles qui facilite et structure le développement d'applications.
Frontend	Partie du projet qui sera visible par l'utilisateur, c'est la partie du code qui affichera le site internet
GIT	Système de gestion de version décentralisé permettant de suivre les modifications d'un projet, de collaborer efficacement à plusieurs et de revenir à des versions antérieures du code ou des fichiers.
HTML	Langage de balisage utilisé pour structurer et organiser le contenu des pages web, en définissant des éléments comme les titres, paragraphes,

	images ou liens.
Image Docker	Fichier léger et autonome contenant tout le nécessaire (code, bibliothèques, dépendances) pour exécuter une application dans un conteneur Docker, garantissant la portabilité et la cohérence entre environnements.
JavaScript	Langage de programmation principalement utilisé pour rendre les pages web interactives en permettant de manipuler le contenu, gérer les événements et communiquer avec des serveurs.
jetons CSRF	Cross-Site Request Forgery, Jetons de sécurité générés pour protéger les applications web contre les attaques par falsification de requêtes intersites en vérifiant que les requêtes proviennent d'utilisateurs légitimes.
JSON	JavaScript Object Notation : Format de données textuel dérivé de la notation des objets du langage JavaScript.
JSON Web Token	Format compact et sécurisé de jeton d'authentification utilisé pour transmettre des informations entre deux parties sous forme d'un objet JSON signé numériquement.
Linter	Outils dans les éditeur de texte qui permet de suivre des règle pour écrire le code, par exemple finir les ligne avec des ";". Commencer les variable sans majuscule...
MariaDB	Système de gestion de base de données relationnelle open source, dérivé de MySQL, offrant robustesse, performance et compatibilité avec les applications SQL.
Material Design	Système de design développé par Google qui définit des principes visuels, d'animation et d'interaction visant à créer des interfaces cohérentes, intuitives et esthétiques sur tous les types d'appareils.
MCD	Représentation graphique abstraite des données d'un système, décrivant les entités, leurs attributs et les relations entre elles, utilisée pour concevoir des bases de données.
Méthode agiles	Ensemble d'approches de gestion de projet centrées sur l'adaptabilité, la collaboration et l'amélioration continue, privilégiant des cycles courts de développement pour répondre rapidement aux besoins changeants des utilisateurs.
Modèle MVC	Modèle-Vue-Contrôleur, Architecture qui sépare les données, l'affichage et la logique de contrôle pour mieux organiser le code.
MySQL	Système de gestion de base de données relationnelle très utilisé pour stocker et organiser des données.
Nginx	Serveur web performant et léger utilisé pour servir des sites web, gérer des requêtes HTTP, faire du proxy inverse, de la répartition de charge ou de la sécurisation d'applications web.
ORM	Technique permettant de convertir des données entre des systèmes incompatibles, en représentant les tables d'une base de données sous forme d'objets dans un langage de programmation.
PHP	Langage de programmation côté serveur principalement utilisé pour

	développer des sites web dynamiques et interagir avec des bases de données.
Pinia	Bibliothèque de gestion d'état pour les applications Vue.js, offrant une approche simple, intuitive et moderne pour centraliser et partager les données entre composants.
Port	Canaux numériques identifiés par un numéro qui permettent aux applications de communiquer entre elles via un réseau, en distinguant les différents services accessibles sur une même machine.
Refactoring	Action de réorganiser ou réécrire le code sans en changer le fonctionnement, afin de le rendre plus propre, lisible et facile à maintenir.
Repository (ORM)	Composant qui centralise les opérations de lecture et d'écriture sur une entité ou un groupe d'entités dans une base de données, facilitant l'abstraction des requêtes.
Routes	Chemins ou URL définis dans une API qui correspondent à des points d'accès spécifiques permettant d'effectuer des actions précises (comme récupérer, créer, modifier ou supprimer des données).
SEO	techniques pour améliorer la visibilité d'un site web sur les moteurs de recherche.
SPA	Applications web qui chargent une seule page HTML et mettent à jour dynamiquement son contenu via JavaScript, offrant une navigation fluide sans rechargement complet de la page.
stateless	Caractéristique d'un système ou service qui ne conserve aucune information sur les interactions passées, traitant chaque requête de manière indépendante et autonome.
Stimulus (JavaScript)	Framework JavaScript léger qui améliore l'interactivité des pages web en connectant facilement du code JavaScript à des éléments HTML via des attributs, sans recharger la page.
Store vue	Structure centralisée qui permet de stocker, gérer et partager l'état (données) d'une application entre plusieurs composants de manière cohérente et réactive.
Symfony	Framework PHP pour développer des applications web robustes et structurées.
Test intégration	Type de test logiciel qui vérifie si plusieurs composants ou modules d'une application fonctionnent correctement ensemble lorsqu'ils sont combinés.
Tests unitaires	Petits tests automatisés qui vérifient que chaque partie du code (souvent une fonction) fonctionne correctement de manière isolée.
Token	Jeton numérique utilisé pour authentifier et sécuriser les échanges entre un utilisateur et un service, souvent sous forme de chaîne codée représentant des informations d'accès.
Trello	Application de gestion de projets basée sur le principe de tableaux, qui permet d'organiser des tâches sous forme de cartes déplaçables, facilitant la collaboration visuelle et le suivi d'avancement.
Twig	Moteur de templates pour PHP, souvent utilisé avec Symfony pour

	générer du HTML proprement.
TypeScript	Langage basé sur JavaScript qui ajoute le typage pour plus de sécurité et de clarté dans le code.
URL	Uniform Resource Locator : Adresse d'un site ou d'une page sur Internet
User Flow	un schéma qui montre les étapes et décisions prises par l'utilisateur pour naviguer dans une application ou un site.
UX	User Expérience, désigne l'ensemble des ressentis et interactions d'un utilisateur lorsqu'il utilise un produit ou un service, comme un site web ou une application.
Vite	Outil de build ultra-rapide pour les projets frontend, souvent utilisé avec Vue ou React.
Vue.js	Framework JavaScript pour créer des interfaces web interactives.
Vuetify	Bibliothèque de composants UI basée sur Vue.js, avec un design Material Design.
Workflow	Enchaînement structuré des tâches, des actions ou des étapes nécessaires à la réalisation d'un processus, souvent automatisé ou modélisé pour améliorer l'efficacité et la coordination dans un projet.
XML	Extensible Markup Language : Métalangage informatique de balisage générique qui est un sous-ensemble du Standard Generalized Markup Language.